

Mutation Testing Coverage Report

A. M Samdani Mozumder

BSSE 1412

Jul 09, 2025

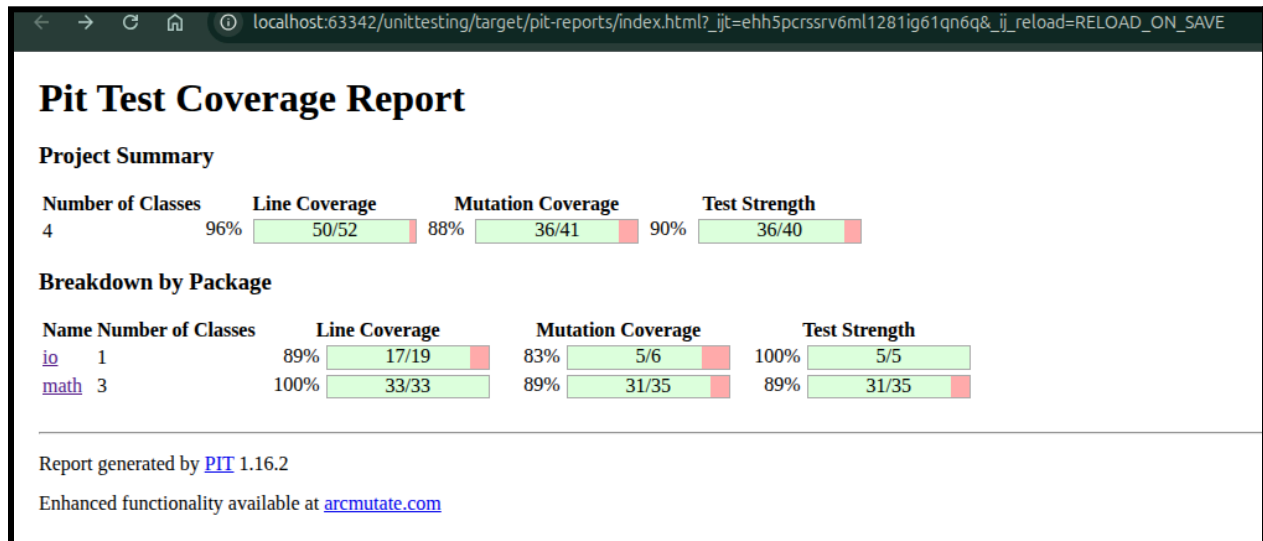


Fig 1: Initial Pit Coverage Report of whole project

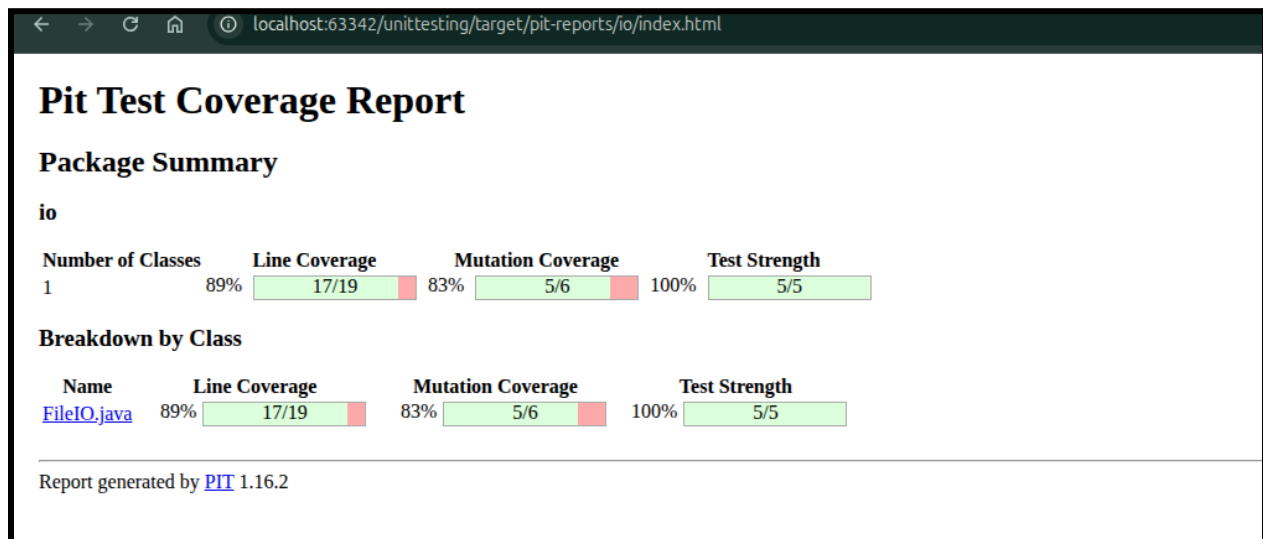


Fig 2: Initial Pit Coverage Report of FileIO class

```

19 public class FileIO {
20
21     /**
22      * Reads a file that contains numbers line by line
23      * and returns an array of the integers found in the file.
24      * @param filepath the file that contains the numbers
25      * @return an array of numbers
26      * @exception IllegalArgumentException when the given file does not exist
27      * @exception IllegalArgumentException when the given file is empty
28      * @exception NumberFormatException for checking invalid entries
29      * @exception IOException when an IO interruption occurs (not required to be tested)
30      */
31     public int[] readFile(String filepath) {
32         File file = new File(filepath);
33         if (!file.exists())
34             throw new IllegalArgumentException("Input file does not exist");
35
36         List<Integer> numbersList = new ArrayList<>();
37         BufferedReader reader;
38         try {
39             reader = new BufferedReader(new FileReader(file));
40             String line = null;
41             while ((line = reader.readLine()) != null) {
42                 try {
43                     int number = Integer.parseInt(line);
44                     numbersList.add(number);
45                 } catch (NumberFormatException e) {
46                     throw new NumberFormatException();
47                     // Do nothing will skip the current invalid line
48                 }
49             }
50         } catch (IOException e) {
51             e.printStackTrace();
52         }
53
54         if (numbersList.size() == 0)
55             throw new IllegalArgumentException("Given file is empty");
56
57         // Convert a List to an array using
58         return numbersList.stream().mapToInt(i -> i).toArray();
59     }
60 }
61

```

Fig 3: Initial line and mutation coverage of FileIO class

Analysis of Uncovered Lines

The report indicates that 17 out of 19 lines in FileIO.java were executed, meaning 2 lines were not covered.

Why These Lines Are Not Covered

- Line 51 (e.printStackTrace();): The test cases do not simulate an IOException. For example, they don't test scenarios like a file being locked, lacking read permissions, or an I/O interruption occurring during reader.readLine(). The Javadoc explicitly states that IOException is "not required to be tested," which explains why the test suite omits this case.
- Second Uncovered Line: If the second line is the closing brace of the IOException catch block or another line in the try-catch structure, it's likely due to the same issue: no test triggers an IOException.

Mutations	
33	1. negated conditional → KILLED
41	1. negated conditional → KILLED
51	1. removed call to java/io/IOException::printStackTrace → NO_COVERAGE
54	1. negated conditional → KILLED
58	1. replaced return value with null for io/FileIO::readFile → KILLED
	2. replaced int return with 0 for io/FileIO::lambda\$readFile\$0 → KILLED

Fig 4: Initial mutations of FileIO class

Analysis of Mutation Coverage

The report shows 5 out of 6 mutants were killed, meaning one mutant(51) survived.

Why This Mutant Survived

- The IOException catch block is not exercised by any test case, as discussed in the line coverage section. The test suite focuses on valid files, empty files, non-existent files, and files with invalid entries, but it does not simulate I/O errors.
- The mutation (removing printStackTrace()) is not detectable by the current tests because printStackTrace() only produces a side effect (console output) and does not alter the method's return value or exception-throwing behavior, which are the primary assertions in the tests.

Pit Test Coverage Report

Package Summary

math

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
3	100% 33/33	89% 31/35	89% 31/35

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
ArithmeticOperations.java	100% 11/11	85% 11/13	85% 11/13
ArrayOperations.java	100% 7/7	100% 5/5	100% 5/5
MyMath.java	100% 15/15	88% 15/17	88% 15/17

Report generated by [PIT](#) 1.16.2

Fig 5: Initial Pit Coverage Report of math package

```

10  */
11  public class ArithmeticOperations {
12
13      /**
14       * Performs the basic arithmetic operation of division.
15       *
16       * @param numerator the numerator of the operation
17       * @param denominator the denominator of the operation
18       * @return the result of the division between numerator and denominator
19       * @exception ArithmeticException when denominator is zero
20       */
21      public double divide(double numerator, double denominator) {
22  1      if (denominator == 0)
23          throw new ArithmeticException("Cannot divide with zero");
24
25  2      return numerator / denominator;
26      }
27
28      /**
29       * Performs the basic arithmetic operation of multiplication between two
30       * positive Integers
31       *
32       * @param x the first input
33       * @param y the second input
34       * @return the product of the multiplication
35       * @exception IllegalArgumentException when <b>x</b> or <b>y</b> are negative
36       *                                     numbers
37       * @exception IllegalArgumentException when the product does not fit in an
38       *                                     Integer variable
39       */
40      public int multiply(int x, int y) {
41  4      if (x < 0 || y < 0) {
42          throw new IllegalArgumentException("x & y should be >= 0");
43  1      } else if (y == 0) {
44          return 0;
45  3      } else if (x <= Integer.MAX_VALUE / y) {
46  2      return x * y;
47      } else {
48          throw new IllegalArgumentException("The product does not fit in an Integer variable");
49      }
50      }
51
52  }

```

Fig 6: Initial line and mutation coverage of ArithmeticOperations class

Mutations

```

22  1. negated conditional - KILLED
25  1. Replaced double division with multiplication - KILLED
    2. replaced double return with 0.0d for math/ArithmeticOperations::divide - KILLED
41  1. negated conditional - KILLED
    2. negated conditional - KILLED
    3. changed conditional boundary - KILLED
    4. changed conditional boundary - KILLED
43  1. negated conditional - KILLED
45  1. Replaced integer division with multiplication - SURVIVED Covering tests
    2. negated conditional - KILLED
    3. changed conditional boundary - SURVIVED Covering tests
46  1. Replaced integer multiplication with division - KILLED
    2. replaced int return with 0 for math/ArithmeticOperations::multiply - KILLED

```

Fig 7: Initial mutations of ArithmeticOperations class

Analysis of Mutation Coverage

The report shows 11 out of 13 mutants were killed, meaning two mutants survived.

- **Lack of Boundary Testing at Exact Limits**

The original test suite focused on cases where the product exceeded `Integer.MAX_VALUE`, triggering the `IllegalArgumentException`. However, it didn't test cases where `x * y` is exactly at or just below the boundary, where the mutation (`<=` to `<`) could change the outcome. Without a test at the precise boundary, the mutated condition's behavior went undetected.

- **Insufficient Coverage of Zero with Large Inputs**

The test `multiply_x_value_zero` and `multiply_y_value_zero` checked small values, but none tested `y = 0` with a large `x` like `Integer.MAX_VALUE`. The mutation could alter the flow if the boundary check was reached inappropriately with large numbers, but the absence of this specific case meant the mutation's impact was not exposed, allowing it to survive.

ArrayOperations.java

```

1  package math;
2
3  import java.util.ArrayList;
4  import java.util.Arrays;
5  import java.util.List;
6
7  import io.FileIO;
8
9  /**
10   * The MyMath provides simple methods such as computing a factorial or finding
11   * whether an integer is prime
12   *
13   * @author pandeliskirpoglou
14   * @version 1.0
15   * @since 2020-04-18
16   */
17
18  public class ArrayOperations {
19
20      /**
21       * Gets one integer and returns true if it is prime and false if it is not.
22       *
23       * @param fileio    instance for reading a file
24       * @param filepath  path for file that needs to be checked
25       * @param myMath    instance for checking whether a number is prime
26       * @return arrayOfPrimeNumbers the array of prime numbers that where in the file
27       */
28
29      public int[] findPrimesInFile(FileIO fileio, String filepath, MyMath myMath) {
30          int[] arrayOfNumbers = fileio.readFile(filepath);
31          List<Integer> arrayOfPrimeNumbers = new ArrayList<>();
32          for (int i = 0; i < arrayOfNumbers.length; i++) {
33              if (myMath.isPrime(arrayOfNumbers[i])) {
34                  arrayOfPrimeNumbers.add(arrayOfNumbers[i]);
35              }
36          }
37          return arrayOfPrimeNumbers.stream().mapToInt(i -> i).toArray();
38      }
39  }
40  }
```

Fig 8: 100% line coverage at initial of ArrayOperations class

Mutations

```

32  1. negated conditional - KILLED
    2. changed conditional boundary - KILLED
33  1. negated conditional - KILLED
37  1. replaced int return with 0 for math/ArrayOperations::lambda$findPrimesInFile$0 - KILLED
    2. replaced return value with null for math/ArrayOperations::findPrimesInFile - KILLED
```

Fig 9: 100% mutation coverage at initial of ArrayOperations class

```

11
12 public class MyMath {
13
14     /**
15      * Gets one integer and returns it's factorial.
16      *
17      * @param n the number of which we want the factorial
18      * @return fact the factorial of the number n
19      * @exception IllegalArgumentException when inputs n < 0 and n > 12
20      */
21
22     public int factorial(int n) {
23         int fact = 1;
24         if (n < 0 || n > 12) {
25             throw new IllegalArgumentException("number should be 0 or above and 12 or below");
26         } else {
27             for (int i = 1; i <= n; i++) {
28                 fact = fact * i;
29             }
30         }
31
32         return fact;
33     }
34
35     /**
36      * Gets one integer and returns true if it is prime and false if it is not.
37      *
38      * @param n the number we are trying to find out whether it is prime or not
39      * @return isPrimeNumber true if n is prime | false if n is not prime
40      * @exception IllegalArgumentException when inputs n < 2
41      */
42
43     public boolean isPrime(int n) {
44         boolean isPrimeNumber = true;
45         if (n < 2) {
46             throw new IllegalArgumentException("No prime numbers below 2");

```

Fig 10: Initial line and mutation coverage of MyMath class

```

58
59 }

Mutations

1. negated conditional → KILLED
2. negated conditional → KILLED
24 3. changed conditional boundary → SURVIVED Covering tests
4. changed conditional boundary → SURVIVED Covering tests
27 1. changed conditional boundary → KILLED
2. negated conditional → KILLED
28 1. Replaced integer multiplication with division → KILLED
32 1. replaced int return with 0 for math/MyMath::factorial → KILLED
45 1. changed conditional boundary → KILLED
2. negated conditional → KILLED
48 1. changed conditional boundary → KILLED
2. negated conditional → KILLED
3. Replaced integer division with multiplication → KILLED
49 1. Replaced integer modulus with multiplication → KILLED
2. negated conditional → KILLED
56 1. replaced boolean return with true for math/MyMath::isPrime → KILLED
2. replaced boolean return with false for math/MyMath::isPrime → KILLED

```

Fig 11: Initial mutation coverage of MyMath class

Analysis of Mutation Coverage

Line 24: changed conditional boundary → SURVIVED (two mutations).

- These mutations involve changing the conditional boundary in the statement if (n < 0 || n > 12).
- The mutations likely changed n < 0 to n <= 0 and n > 12 to n >= 12, and they survived because the test suite does not check the boundary conditions at n = 0 and n = 12.

After upgrading

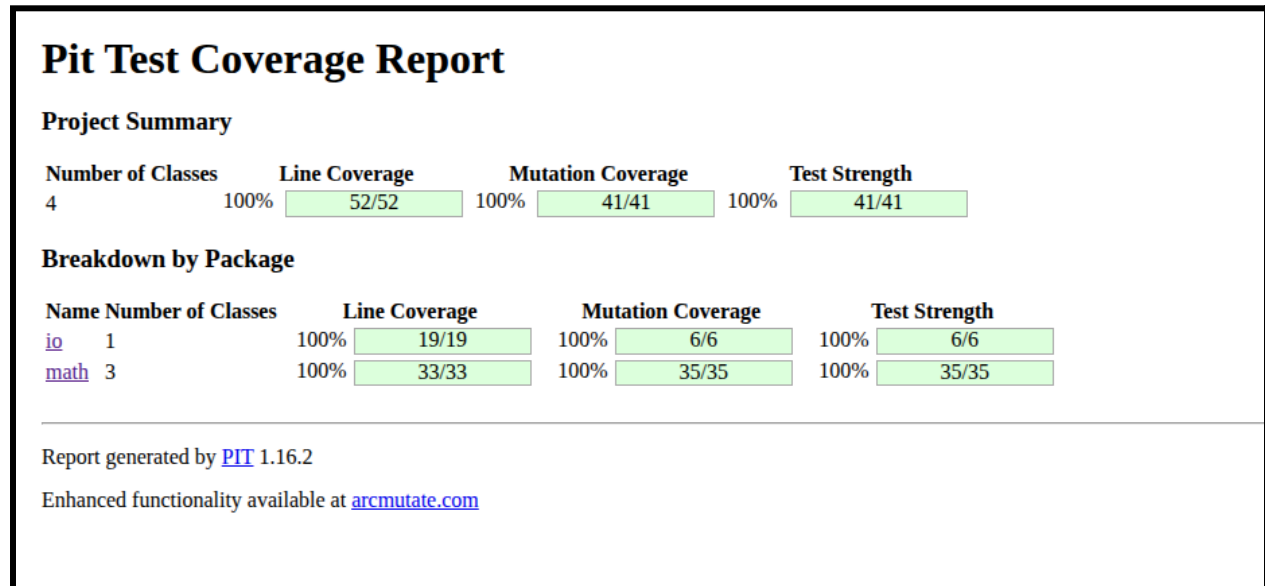


Fig 12: Final Pit Coverage Report of whole project

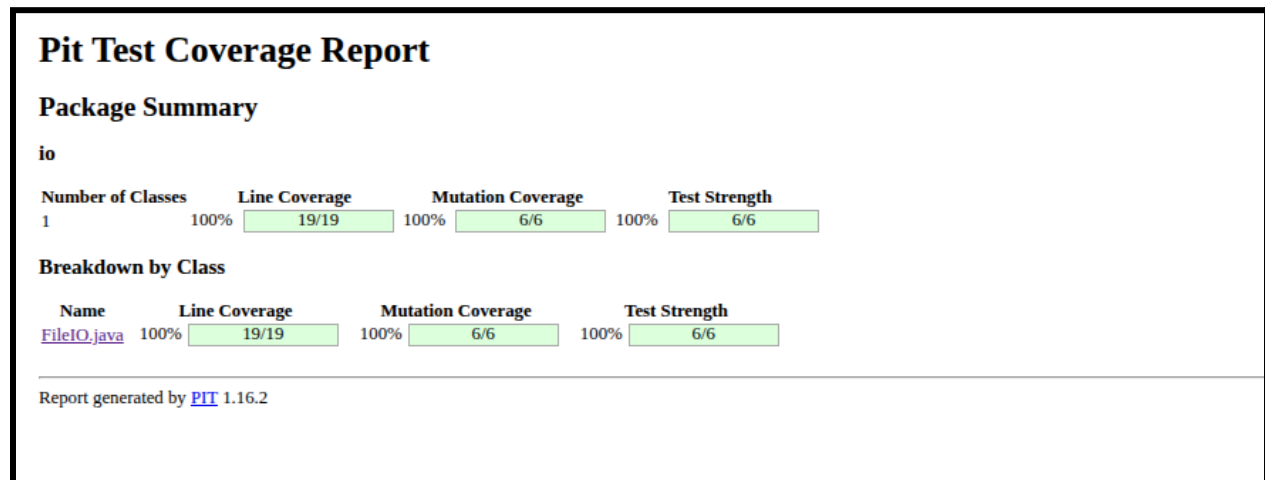


Fig 13: Final Pit Coverage Report of FileIO class

```

49         }
50     } catch (IOException e) {
51         e.printStackTrace();
52     }
53
54     if (numbersList.size() == 0)
55         throw new IllegalArgumentException("Given file is empty");
56
57     // Convert a List to an array using
58     return numbersList.stream().mapToInt(i -> i).toArray();
59 }
60
61 }

```

Mutations

```

33 1. negated conditional -> KILLED
41 1. negated conditional -> KILLED
51 1. removed call to java/io/IOException::printStackTrace -> KILLED
54 1. negated conditional -> KILLED
58 1. replaced return value with null for io/FileIO::readFile -> KILLED
    2. replaced int return with 0 for io/FileIO::lambda$readFile$0 -> KILLED

```

Fig 14: Final mutation coverage of FileIO class

```

@Test new *
void test_io_exception() {
    File tempFile = new File( pathname: "src/test/resources/restricted_file.txt");
    try {
        tempFile.createNewFile();
        tempFile.setReadable(false);
        assertThrows(IllegalArgumentException.class, () -> {
            fileio.readFile(tempFile.getPath());
        });
        String errOutput = errContent.toString();
        assertTrue( condition: errOutput.contains("java.io.FileNotFoundException") ||
                    errOutput.contains("Permission denied"),
                    message: "Expected FileNotFoundException stack trace in System.err output");
    } catch (IOException e) {
        e.printStackTrace();
        fail("Failed to create test file: " + e.getMessage());
    } finally {
        tempFile.setReadable(true);
        tempFile.delete();
    }
}

```

Fig 15: New test case for kill that mutant

Why This Kills the Mutant

- When PIT applies the mutant (removes `e.printStackTrace()`), no stack trace is printed to `System.err`. The `errContent.toString()` will be empty or lack the expected `FileNotFoundException` string, causing the `assertTrue` to fail. This detects the mutation, killing the mutant.
- Mutant at Line 51: The removed call to `java/io/IOException::printStackTrace` mutant is now killed because the test explicitly checks for the presence of the stack trace in `System.err`. If the call is removed, the `assertTrue` condition fails, causing the test to fail and the mutant to be killed.

Pit Test Coverage Report

Package Summary

math

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
3	100% 33/33	100% 35/35	100% 35/35

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
ArithmeticOperations.java	100% 11/11	100% 13/13	100% 13/13
ArrayOperations.java	100% 7/7	100% 5/5	100% 5/5
MyMath.java	100% 15/15	100% 17/17	100% 17/17

Report generated by [PIT](#) 1.16.2

Fig 16: Final Pit Coverage Report of math package

```
10  */
11  public class ArithmeticOperations {
12
13      /**
14       * Performs the basic arithmetic operation of division.
15       *
16       * @param numerator the numerator of the operation
17       * @param denominator the denominator of the operation
18       * @return the result of the division between numerator and denominator
19       * @exception ArithmeticException when denominator is zero
20       */
21      public double divide(double numerator, double denominator) {
22          1 if (denominator == 0)
23              throw new ArithmeticException("Cannot divide with zero");
24
25          2 return numerator / denominator;
26      }
27
28      /**
29       * Performs the basic arithmetic operation of multiplication between two
30       * positive Integers
31       *
32       * @param x the first input
33       * @param y the second input
34       * @return the product of the multiplication
35       * @exception IllegalArgumentException when <b>x</b> or <b>y</b> are negative
36       *         numbers
37       * @exception IllegalArgumentException when the product does not fit in an
38       *         Integer variable
39       */
40      public int multiply(int x, int y) {
41          4 if (x < 0 || y < 0) {
42              throw new IllegalArgumentException("x & y should be >= 0");
43          } else if (y == 0) {
44              return 0;
45          } else if (x <= Integer.MAX_VALUE / y) {
46              3 return x * y;
47          } else {
48              2 throw new IllegalArgumentException("The product does not fit in an Integer variable");
49          }
50      }
51  }
52 }
```

Fig 17: Final line and mutation coverage of ArithmeticOperations class

Mutations	
22	1. negated conditional → KILLED
25	1. Replaced double division with multiplication → KILLED 2. replaced double return with 0.0d for math/ArithmeticOperations::divide → KILLED
41	1. negated conditional → KILLED 2. negated conditional → KILLED 3. changed conditional boundary → KILLED 4. changed conditional boundary → KILLED
43	1. negated conditional → KILLED
45	1. Replaced integer division with multiplication → KILLED 2. negated conditional → KILLED 3. changed conditional boundary → KILLED
46	1. Replaced integer multiplication with division → KILLED 2. replaced int return with 0 for math/ArithmeticOperations::multiply → KILLED

Fig 18: All mutation are killed of ArithmeticOperations class

```

@Test new *
void multiply_y_value_zero_with_large_x() {
    int expected = 0;
    int actual = arithmeticOperations.multiply(Integer.MAX_VALUE, y: 0);
    assertEquals(expected, actual);
}

@Test new *
void test_multiply__max_boundary() {
    int maxDiv2 = Integer.MAX_VALUE / 2;
    assertEquals( expected: maxDiv2 * 2, arithmeticOperations.multiply(maxDiv2, y: 2));
}

```

Fig 19: New test cases for kill mutants

Why This Kills the Mutants

1.multiply_y_value_zero_with_large_x

- Tests multiply(Integer.MAX_VALUE, 0) with an expected result of 0.
- In the original code, y = 0 triggers the y == 0 condition, returning 0. The mutation (<= to <) doesn't affect this path since the check x <= Integer.MAX_VALUE / y isn't reached (division by zero is avoided).
- This kills the mutation by ensuring the early return logic holds, exposing any boundary change that might misroute the execution.

2.multiply_max_boundary

- Tests multiply(maxDiv2, 2) where maxDiv2 = Integer.MAX_VALUE / 2, expecting Integer.MAX_VALUE.
- In the original code, maxDiv2 * 2 <= Integer.MAX_VALUE / 2 is true (since maxDiv2 = 1073741823, and 1073741823 * 2 = 2147483646 <= 2147483647), returning the product. In the mutated code (<), maxDiv2 < Integer.MAX_VALUE / 2 is false (since 1073741823 equals Integer.MAX_VALUE / 2), potentially throwing an exception or failing.
- This exposes the boundary difference, killing the mutation.

```

12 public class MyMath {
13
14     /**
15      * Gets one integer and returns it's factorial.
16      *
17      * @param n the number of which we want the factorial
18      * @return fact the factorial of the number n
19      * @exception IllegalArgumentException when inputs n < 0 and n > 12
20      */
21
22     public int factorial(int n) {
23         int fact = 1;
24         if (n < 0 || n > 12) {
25             throw new IllegalArgumentException("number should be 0 or above and 12 or below");
26         } else {
27             for (int i = 1; i <= n; i++) {
28                 fact = fact * i;
29             }
30         }
31
32         return fact;
33     }

```

Fig 20: Final line and mutation coverage of MyMath class

Mutations

```

24 1. negated conditional → KILLED
2. negated conditional → KILLED
3. changed conditional boundary → KILLED
4. changed conditional boundary → KILLED
27 1. changed conditional boundary → KILLED
2. negated conditional → KILLED
28 1. Replaced integer multiplication with division → KILLED
32 1. replaced int return with 0 for math/MyMath::factorial → KILLED
45 1. changed conditional boundary → KILLED
2. negated conditional → KILLED
48 1. changed conditional boundary → KILLED
2. negated conditional → KILLED
3. Replaced integer division with multiplication → KILLED
49 1. Replaced integer modulus with multiplication → KILLED
2. negated conditional → KILLED
56 1. replaced boolean return with true for math/MyMath::isPrime → KILLED
2. replaced boolean return with false for math/MyMath::isPrime → KILLED

```

Fig 21: Final mutation coverage(100%) of MyMath class

```

@Test new *
void factorial_zero() {
    int expected = 1;
    int actual = myMath.factorial(n: 0);
    assertEquals(expected, actual);
}

@Test new *
void factorial_twelve() {
    int expected = 479001600;
    int actual = myMath.factorial(n: 12);
    assertEquals(expected, actual);
}

```

Fig 22: New test cases for kill mutants

Why This Kills the Mutants

- **factorial_zero:** Tests the boundary condition $n = 0$, where the expected factorial is 1. This should kill the surviving mutation that changes $n < 0$ to $n \leq 0$, as it ensures the method behaves correctly at the lower boundary.
- **factorial_twelve:** Tests the boundary condition $n = 12$, where the expected factorial is 479001600. This should kill the surviving mutation that changes $n > 12$ to $n \geq 12$, as it ensures the method behaves correctly at the upper boundary.

No changes are needed for the `isPrime` method, as all its mutations were killed according to the report.